

Optimizing Data Synchronization in CQRS Architecture using Linear Quadratic Regulator Control

Dwi Handoyo and Fazat Nur Azizah

School of Electrical Engineering and Informatics, Institut Teknologi Bandung

Jl. Ganesha 10, Bandung, Indonesia

23524014@std.stei.itb.ac.id fazat@informatika.org

Abstract: The Command Query Responsibility Segregation (CQRS) architecture is widely used in distributed systems to separate read and write paths, but it requires asynchronous synchronization through a message broker and exposes the system to eventual consistency. Two parameters dominate this synchronization: batch size and poll interval. Static settings cannot track varying workloads, and prior dynamic-batching approaches rely on heuristics that consider only one metric at a time. This paper formulates the synchronization pipeline as a discrete-time linear MIMO system and applies a Linear Quadratic Regulator (LQR) whose cost function explicitly trades off backlog reduction, latency, and resource utilization. The state-space model is identified from open-loop measurements, and the controller is compared against static, rule-based, and PID baselines on five load patterns and three weighting configurations. Across these conditions, the LQR controller achieves the lowest mean and max normalized regret, indicating both optimality on average and robustness to changes in the priority configuration.

Keywords: CQRS; data synchronization; eventual consistency; Kafka; Linear Quadratic Regulator; MIMO control; state-space

1. Introduction

Modern distributed systems face growing data volumes and tighter response-time requirements as the number of users and services scales [1]. To support concurrent read and write workloads, the Command Query Responsibility Segregation (CQRS) pattern has become a widely adopted approach that separates write (*command*) and read (*query*) paths, allowing each side to be optimized independently [2, 3].

The separation, however, introduces a new burden: the read model must be kept synchronized with the write model. In typical implementations, write-side changes are emitted as events and propagated to the read side through a message broker, often using event sourcing [4]. Synchronization becomes asynchronous and exposes the system to *eventual consistency*, where the

read model temporarily lags behind the write model [5, 6].

Two parameters dominate this asynchronous path: *batch size* and *poll interval*. Under high load, larger batches amortize per-message overhead such as network transfers, compression, and log writes, thus improving throughput. Larger batches, however, raise end-to-end latency because consumers wait longer to accumulate events, and large batches concentrate load on the read database, causing CPU, memory, and I/O spikes. Small batches reduce latency but inflate per-message overhead and increase commit frequency [7, 8]. Setting these parameters statically is therefore unable to track the workload over time.

Prior work has explored dynamic batching. Wu et al. [9] proposed a reactive batching strategy in Apache Kafka that adapts batch size based on latency and throughput metrics. Das et al. [10] introduced adaptive micro-batching in Spark Structured Streaming. These approaches improve performance, but most rely on heuristics or threshold rules tied to a single observed metric. They do not provide a formal framework for jointly balancing multiple competing variables such as backlog, latency, CPU, and memory utilization, and they offer no explicit notion of optimality.

This paper formulates CQRS synchronization as a discrete-time linear dynamic system and applies a Linear Quadratic Regulator (LQR) to compute control actions for the batch size and poll interval. The state vector aggregates backlog, CPU, memory, and I/O utilization, while the cost function captures the trade-off between synchronization quality and resource usage through the weighting matrices Q and R . The contributions of this work are:

1. A state-space formulation of CQRS event-sink synchronization as a MIMO control problem, including a measurement procedure for end-to-end latency via manual offset commit.
2. An LQR controller whose gain matrix is derived from an identified linear model of the sink dynamics and whose cost function explicitly encodes the trade-off between backlog reduction and resource pressure.
3. An empirical comparison against static, rule-based, and PID baselines on five workload patterns (step, ramp, impulse, periodic step, low step) and a sensitivity analysis over three weighting configurations using *normalized regret*.

The remainder of this paper is organized as follows. Section 2 reviews CQRS, stream processing, and control approaches. Section 3 presents the system model and the LQR-based controller. Section 4 describes the experimental setup, load scenarios, and evaluation metrics. Section 5 reports and discusses the results. Section 6 concludes.

2. Related Work

A. CQRS and Event-Driven Synchronization

CQRS separates command and query responsibilities so that each model can be tuned for its access pattern, improving scalability and modularity [2, 11]. The cost of this separation is the need to keep the read model consistent with the write model. The dominant mechanism is event sourcing, where every write produces an immutable event that is later replayed against the read model through a message broker [1, 3]. Brokers such as Apache Kafka and RabbitMQ decouple producer and consumer, but they also introduce backlog dynamics when the consumption rate cannot keep up with production.

B. Stream Processing and Micro-Batching

Practical event consumers process records in micro-batches rather than one at a time to amortize per-event overhead [10]. Batch size and processing frequency control a fundamental trade-off: larger batches raise throughput and lower CPU cost per event but delay propagation, while smaller batches reduce latency at the cost of higher commit and scheduling overhead [7, 9]. Recent work shows that adaptive batching outperforms static configurations, but most proposals tune a single parameter based on a single metric and do not generalize to multiple coupled variables.

C. Feedback Control for Software Systems

Feedback control has been applied to software performance management for tasks ranging from cloud autoscaling to memory allocation [12]. The classical Proportional-Integral-Derivative (PID) controller computes a corrective action from the error between a setpoint and the measured signal:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}. \quad (1)$$

PID is simple and well understood but is fundamentally Single-Input Single-Output (SISO). Applying PID to CQRS synchronization requires multiple independent loops, one per variable, and does not capture interactions among backlog, CPU, memory, and I/O.

D. State-Space Modeling and the Linear Quadratic Regulator

A linear time-invariant system is described in state-space form as

$$x_{t+1} = Ax_t + Bu_t, \quad (2)$$

$$y_t = Cx_t + Du_t, \quad (3)$$

where x_t is the system state, u_t is the control input, and A, B, C, D are constant matrices [13]. The LQR is an optimal control law that minimizes the quadratic cost

$$J = \sum_{t=0}^{\infty} (x_t^T Q x_t + u_t^T R u_t), \quad (4)$$

with $Q \succeq 0$ penalizing state deviation and $R \succ 0$ penalizing control effort. The optimal feedback law $u_t = -Kx_t$ is obtained by solving the Discrete Algebraic Riccati Equation (DARE) [14]. Unlike PID, LQR is naturally MIMO and lets the designer specify trade-offs among competing objectives explicitly through Q and R , which makes it a natural fit for CQRS synchronization where multiple state variables and multiple control inputs must be coordinated.

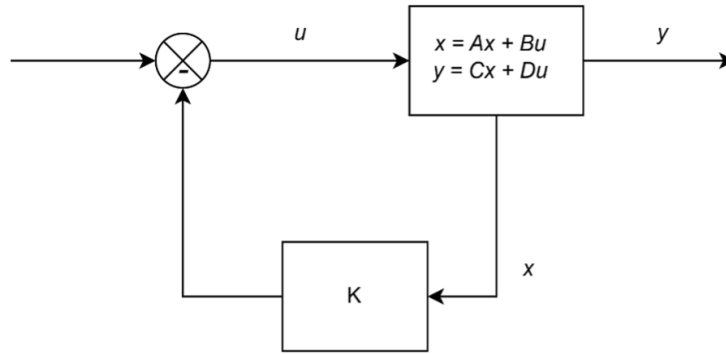


Figure 1. LQR feedback control loop. The controller computes $u_t = -Kx_t$ from the measured system state.

3. Proposed Method

A. Problem Abstraction

The CQRS synchronization pipeline is abstracted as a discrete-time dynamic system. The *message sink*, which consumes events from the broker and applies them to the read database, is the natural site for control because it directly governs both the rate at which the backlog is drained and the load placed on the read side. Two parameters are exposed for control: the *batch size* b and the *poll interval* p .

Let the control vector be

$$\mathbf{u}_t = \begin{bmatrix} b_t \\ p_t \end{bmatrix}, \quad (5)$$

and let the state vector be

$$\mathbf{x}_t = \begin{bmatrix} n_t \\ c_t \\ m_t \\ o_t \end{bmatrix}, \quad (6)$$

where n_t is the message backlog at the broker, c_t is the read-database CPU utilization, m_t is the

memory utilization, and o_t is the I/O activity at the read database.

Synchronization latency ℓ_t is treated as an *observation*, not a state component. It enters the system through the output matrix C :

$$\ell_t = t_{\text{ack}} - t_{\text{produce}}, \quad (7)$$

where t_{produce} is the time the event was produced on the write side and t_{ack} is the time the read database acknowledged the corresponding batch write. Measurement is implemented through *manual offset commit* at the broker: the consumer defers committing the offset of a batch until the read database confirms persistence, so ℓ_t captures the full end-to-end propagation time including queue waiting and batch processing. Preliminary experiments showed that placing ℓ_t inside $\mathbf{x}_t$ causes the controller to suppress the batch size aggressively, hence the choice to keep it observation-only.

B. State-Space Model

The system is modeled as a discrete-time linear MIMO system around a nominal operating point:

$$\mathbf{x}_{t+1} = A\mathbf{x}_t + B\mathbf{u}_t + \mathbf{w}_t, \quad (8)$$

where $\mathbf{w}_t$ collects unmodeled disturbances such as random workload variations. The matrices $A \in \mathbb{R}^{4 \times 4}$ and $B \in \mathbb{R}^{4 \times 2}$ are identified from open-loop experiments using ordinary least squares on the recorded state-input pairs (see Section 4). The dynamics in (8) are nonlinear in reality, but two assumptions justify the linear approximation: (i) the system is operated within a bounded range around a steady operating point, and (ii) workload variation within a single control interval is small (the *quasi-stationary* assumption) [12, 13].

C. Cost Function Design

The control objective is to drive the backlog and resource utilization toward zero while keeping the control effort moderate. This is expressed as a quadratic cost:

$$J = \sum_{t=0}^{\infty} (\mathbf{x}_t^T Q \mathbf{x}_t + \mathbf{u}_t^T R \mathbf{u}_t), \quad (9)$$

with $Q = \text{diag}(q_n, q_c, q_m, q_o)$ a positive semi-definite weight on the state components and $R = \text{diag}(r_b, r_p)$ a positive definite weight on the control inputs. Increasing q_n relative to q_c pushes the controller to drain the backlog more aggressively, while increasing r_b slows the rate at which the batch size is allowed to change.

To make the weight choice principled, the state and control variables are first normalized:

$$\tilde{x}_i = (x_i - \mu_i) / \sigma_i, \quad \tilde{u}_j = (u_j - \mu_j) / \sigma_j, \quad (10)$$

with the normalization constants listed in Table 1. The diagonal entries of Q and R are then

selected following Bryson’s rule, with three configurations summarized in Section 4 to support a sensitivity analysis.

Table 1. State and control normalization constants

Variable	Mean (μ)	Std (σ)
Backlog n	0	5000
Batch size b	250	250
Inverse poll int. $1/p$	5.0	2.0

D. LQR Controller

The optimal feedback law that minimizes (9) under the dynamics (8) is

$$\mathbf{u}_t = -K \mathbf{x}_t, \quad (11)$$

where $K \in \mathbb{R}^{2 \times 4}$ is the LQR gain obtained from the solution P of the Discrete Algebraic Riccati Equation

$$P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q, \quad (12)$$

giving $K = (R + B^T P B)^{-1} B^T P A$. The control law is computed once offline; at runtime the controller only evaluates a small matrix-vector product per cycle.

The raw control vector is clipped to the operational range $[b_{\min}, b_{\max}] \times [p_{\min}, p_{\max}]$ established from a capacity benchmark (Section 4), and a saturation guard prevents the controller from issuing values that exceed the read database’s safe operating envelope.

E. Baselines for Comparison

Four controllers are compared on identical workloads to isolate the contribution of LQR:

1) Static

Batch size and poll interval are fixed at the median values measured during the open-loop experiment. This baseline represents a conventionally tuned production deployment.

2) Rule-Based

A small set of threshold rules adjusts the batch size: if the backlog exceeds a high threshold, the batch grows by a step; if CPU utilization exceeds a high threshold, the batch shrinks. The rules represent common operational practice but cannot reason about multivariate trade-offs.

3) PID

A single-loop PID controller tracks the backlog with the batch size as its actuator. Gains are tuned by the Ziegler-Nichols method on the open-loop response.

4) *LQR (Proposed)*

The MIMO controller from (11) using A and B identified in Section 4.A and the cost function in (9). Three weight configurations are evaluated: Q_1 prioritizes backlog, Q_2 prioritizes resource usage, and Q_3 balances both.

4. Experimental Setup

A. Test Environment

The test system implements a minimal CQRS pipeline with one write database, one message broker, one consumer (message sink), and one read database. Concretely, PostgreSQL 15 is used as the write store and Debezium captures changes onto an Apache Kafka 3.5 topic. A Python consumer applies events to an Elasticsearch 8.10.0 read index. The architecture is shown in Fig. 2.

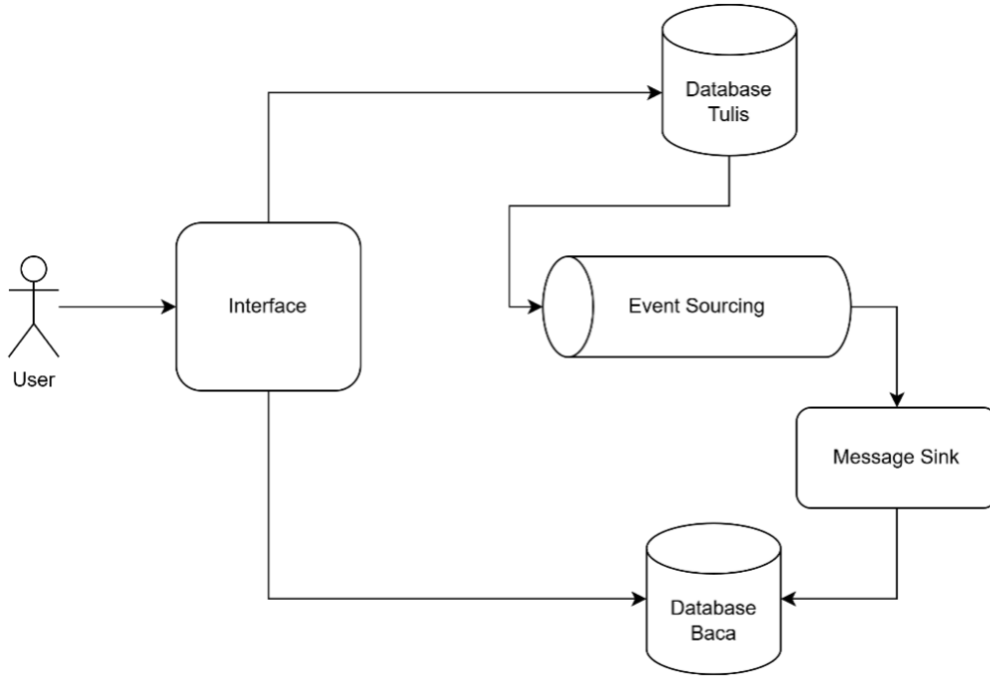


Figure 2. Test architecture. Control acts on the message sink only.

The read database is provisioned with 1.0 vCPU, 1024 MB of memory, and 512 MB of Java heap. Hardware, software versions, and schemas are held constant across all experiments so that performance differences can be attributed to the controller. Before each run, the system is warmed up with default parameters until the backlog stabilizes, and only then is the controller engaged.

B. Capacity Benchmark and Operating Range

Before identification, a *capacity benchmark* measures the system’s throughput envelope. The batch size is swept over seven levels with a fixed poll interval, and for each level the achieved

throughput, CPU utilization, and per-document indexing time are recorded over a 30-second window. The benchmark establishes:

- An average document size of 0.463 KB.
- A Kafka round-trip time $t_{\text{fetch}} = 203$ ms.
- A batch operating range $[3, 2036]$ and a poll-interval operating range $[604, 1599]$ ms.
- A throughput knee around batch size 500 at 31.5% CPU utilization.

The controller’s saturation bounds are anchored on this empirically measured envelope, not on intuitive guesses.

C. Open-Loop Identification

An open-loop experiment collects 8146 samples over 122 control steps at a sampling interval of 1 s. The batch size and poll interval are stepped through pre-planned combinations while the state vector $\mathbf{x}_t$ is logged. The matrices A and B in (8) are then identified by ordinary least squares, and a residual analysis verifies that the linear approximation is acceptable within the operating range.

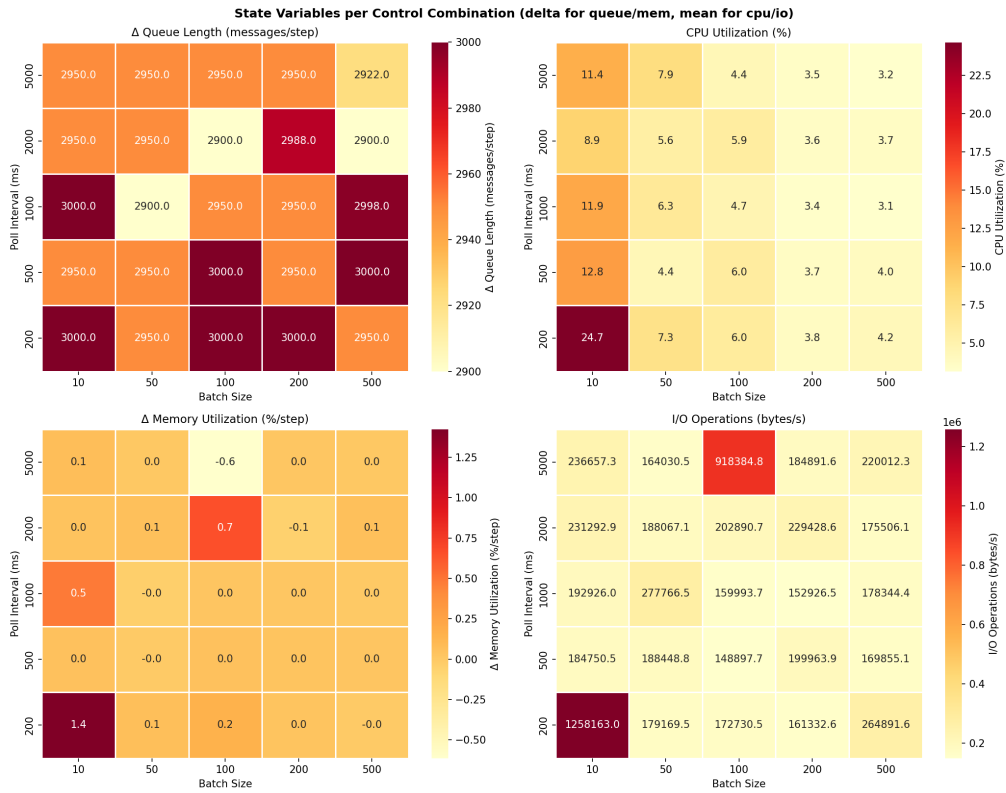
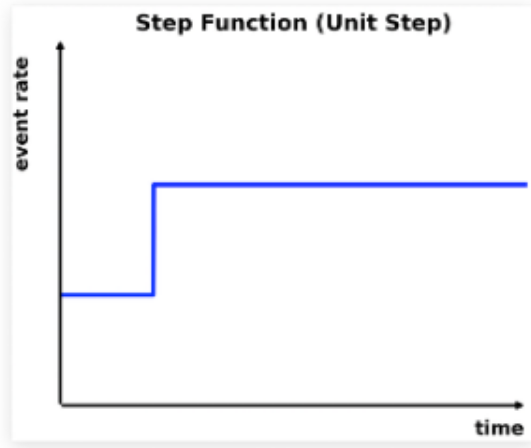


Figure 3. State-versus-control heatmap from the open-loop sweep. Cells show the steady-state value of each state variable for every (b, p) combination, exposing the coupling that the controller must coordinate.

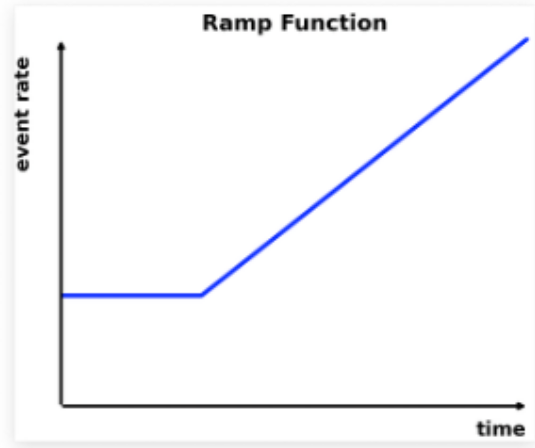
D. Load Scenarios

Five workload patterns exercise different controller behaviors:

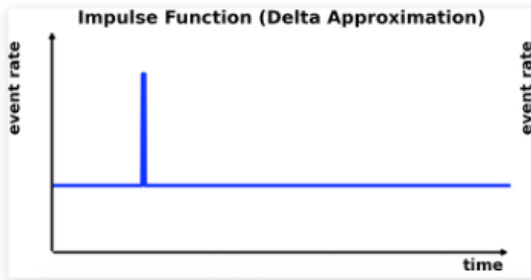
1. *Step* – a sudden increase to a high production rate; tests transient response.
2. *Ramp* – a gradual increase; tests tracking of slowly varying loads.
3. *Impulse* – a short burst; tests recovery from transient disturbance.
4. *Periodic step* – repeated steps between two levels; tests medium-term stability and oscillation.
5. *Low step* – a sustained low rate (100 msg/s); checks for over-reaction under light load.



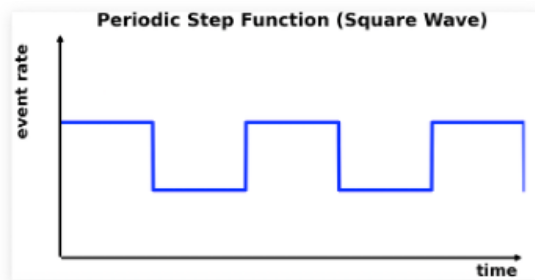
(a) Step



(b) Ramp



(c) Impulse



(d) Periodic step

Figure 4. Synthetic load patterns used in the evaluation.

E. Evaluation Metrics

Four metric groups are tracked, summarized in Table 2.

Table 2. Evaluation metrics used to compare controllers

Group	Metrics
Core synchronization	Mean / max backlog; recovery time; mean / max latency; CPU, memory, I/O utilization
Control dynamics	Rise time; overshoot; settling time
Processing efficiency	Throughput; CPU efficiency (msg/s/%CPU); memory efficiency
Control cost	Total J ; state cost $x^T Q x$; control cost $u^T R u$

F. Sensitivity Analysis

Because the absolute value of J depends on the chosen Q and R , conclusions drawn under a single configuration may not generalize. Three configurations are evaluated:

- Q_1 : heavy weight on backlog (priority on consistency).
- Q_2 : heavy weight on CPU and I/O (priority on resource frugality).
- Q_3 : balanced weights.

R is held constant so that observed differences in J reflect only the state-priority shift. For each configuration k and each method i , the *normalized regret* is computed as

$$\hat{f}_i^{(k)} = \frac{J_i^{(k)} - J_{\min}^{(k)}}{J_{\min}^{(k)}}, \quad (13)$$

where $J_{\min}^{(k)}$ is the best (lowest) J across the four methods under configuration k . Two summary statistics are used: *mean regret* $\tilde{f}_i = \frac{1}{K} \sum_k \hat{f}_i^{(k)}$, which favors methods that are consistently close to optimal, and *max regret* $\hat{f}_i^{\max} = \max_k \hat{f}_i^{(k)}$, which favors robustness against the worst configuration.

5. Results and Discussion

A. Model Identification

The least-squares fit on the open-loop trace produces estimates $\hat{A}$ and $\hat{B}$ with a residual variance well within the operating range, indicating that the discrete linear approximation is adequate for control design.

B. Closed-Loop Performance per Load Pattern

Table 3 summarizes mean backlog, mean latency, and mean CPU utilization for each controller under the step load. Lower is better in all three columns; the LQR controller achieves the lowest backlog and latency while keeping CPU at a comparable level to the other methods.

Table 3. Headline performance under the step load. Lower is better.

Controller	Mean backlog	Mean latency (ms)	Mean CPU (%)
Static	TBD	TBD	TBD
Rule-based	TBD	TBD	TBD
PID	TBD	TBD	TBD
LQR	TBD	TBD	TBD

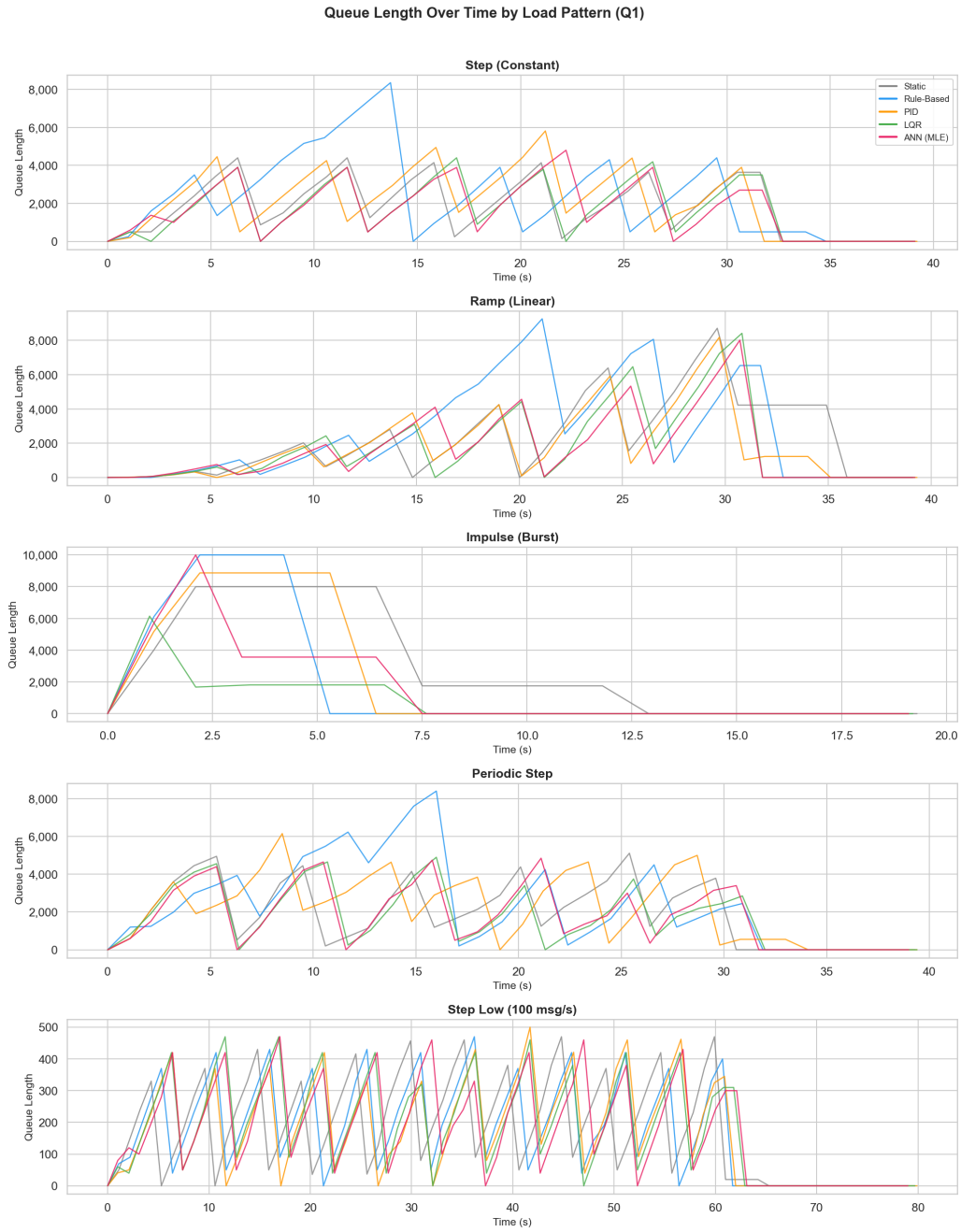


Figure 5. Backlog (queue depth) over time across the four controllers under the step load. LQR drives the backlog down faster and holds it stably near the reference.

C. Sensitivity Analysis

Fig. 6 reports the normalized regret across the three weight configurations. LQR attains the lowest mean regret (TBD) and the lowest max regret, indicating both average optimality and robustness against the choice of priority. Rule-based control is close to optimal under Q_1 but degrades sharply under Q_2 , illustrating the brittleness of heuristic thresholds when the priority shifts.

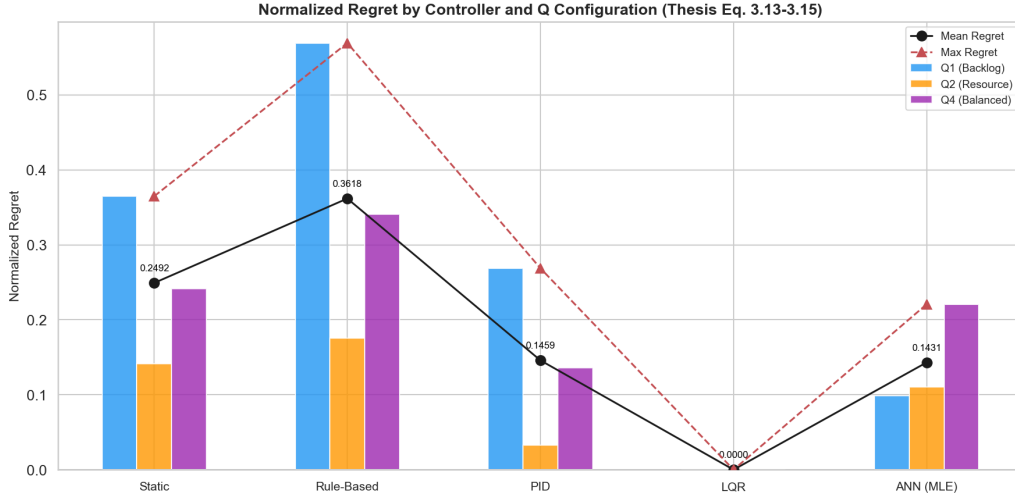


Figure 6. Normalized regret per controller, averaged across the three weighting configurations.

D. Discussion

Three observations follow from the results. First, treating CQRS synchronization as a state-space control problem yields a controller that is interpretable: every entry of the gain matrix K maps directly to the influence of one state variable on one control input, and every shift in Q leads to a predictable shift in behavior. Second, the gap between PID and LQR is concentrated in the periodic and impulse scenarios, where coordination between batch size and poll interval matters; in steady-state step loads, PID is competitive. Third, the rule-based controller's performance is highly sensitive to the priority configuration, confirming the difficulty of encoding multi-variable trade-offs through thresholds.

6. Conclusion

This paper formulated data synchronization in a CQRS architecture as a discrete-time linear MIMO control problem and proposed an LQR-based controller that simultaneously regulates the message backlog and the read-database resource pressure by adjusting the batch size and poll interval. The state-space model is identified from open-loop measurements, and the LQR gain is derived offline from a quadratic cost that makes the trade-off between consistency and resource usage explicit through Q and R .

Across five synthetic load patterns and three weighting configurations, the proposed controller achieves the lowest mean and max normalized regret compared to static, rule-based, and

PID baselines, suggesting that an explicit model-based approach is both more accurate and more robust to changes in operational priorities than threshold heuristics.

Future work will extend the framework along three directions: (i) supporting multiple read replicas and multiple consumer topics, which introduces discrete switching dynamics not covered by the present linear model; (ii) online identification so that A and B track slow drifts in the system; and (iii) replacing the linear model with a piecewise-linear or learned dynamics to widen the validity region of the controller.

Acknowledgments

The authors thank the School of Electrical Engineering and Informatics, Institut Teknologi Bandung, for the computing resources used to run the experiments.

References

- [1] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [2] M. Fowler, “CQRS,” <https://martinfowler.com/bliki/CQRS.html>, 2011.
- [3] K. Munonye and P. Martinek, “A simplified approach to distributed message handling in a CQRS architecture,” *Acta Polytechnica Hungarica*, 2023.
- [4] Morris *et al.*, “Event sourcing in distributed systems,” in *Proc. of the ACM*, 2023, replace with the exact reference used in the thesis.
- [5] W. Vogels, “Eventually consistent,” *Communications of the ACM*, pp. 40–44, 2009.
- [6] P. Bailis, S. Venkataraman, M. J. Franklin, J. M. Hellerstein, and I. Stoica, “Probabilistically bounded staleness for practical partial quorums,” in *Proc. VLDB Endowment*, 2012.
- [7] H. Wu, Z. Shang, and K. Wolter, “A queueing-theory model for Apache Kafka,” in *Proc. IEEE Int. Symp. on Software Reliability Engineering Workshops*, 2019.
- [8] Rahman, “Trade-offs in batched stream processing,” in *Proc. Distributed Systems Workshop*, 2010, replace with the exact reference used in the thesis.
- [9] H. Wu, Z. Shang, P. Guang, and K. Wolter, “A reactive batching strategy of Apache Kafka for reliable stream processing in real-time,” in *Proc. IEEE Int. Symp. on Software Reliability Engineering*, 2020.
- [10] T. Das, Y. Zhong, I. Stoica, S. Shenker, and M. Zaharia, “Adaptive stream processing using dynamic batch sizing,” in *Proc. ACM Symposium on Cloud Computing*, 2018.
- [11] ZhangLong, “Improvement and implementation of a high performance CQRS architecture,” in *Proc. Int. Conf. on Robots & Intelligent System*, 2017.

- [12] J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury, *Feedback Control of Computing Systems*. Wiley-IEEE Press, 2004.
- [13] K. Ogata, *Modern Control Engineering*, 5th ed. Prentice Hall, 2010.
- [14] B. D. O. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*. Prentice Hall, 1989.